

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent	12407519
Kind Code	B2
Date of Patent	September 02, 2025
Inventor(s)	Seibel; James et al.

Trusted execution environment

Abstract

A trusted execution environment on a computing device within an enterprise, whether owned by the enterprise or the employee/user, allows invocation of trusted enterprise applications without hindering external or non-enterprise apps from running on the same computing device. Each of the trusted apps can interact with other trusted apps on the same enterprise computing device in a trusted manner such that other apps or untrusted network connections are prevented for access to the trusted apps. The computing device, however, also executes non enterprise applications which operate independently of the enterprise apps in the same address space using the same unmodified operating system as the enterprise apps on the computing device. The trusted execution environment therefore restricts interprocess communication to be only within the set of enterprise apps and also permits unimpeded operation of other apps under the same OTS (off the shelf) operating system.

Inventors: Seibel; James (Boston, MA), LaFlamme; Kevin (Boston, MA), Koschara; Fred (Boston, MA), Schumak; Reinhard (Boston, MA), Debate; Jeremy (Boston, MA)

Applicant: MALIKIE INNOVATIONS LIMITED (Dublin, IE)

Family ID: 57882006

Assignee: Malikie Innovations Limited (Dublin, IE)

Appl. No.: 17/869298

Filed: July 20, 2022

Prior Publication Data

Document Identifier	Publication Date
US 20220368530 A1	Nov. 17, 2022

Related U.S. Application Data

Publication Classification

Int. Cl.: **H04L9/32** (20060101); **G06F21/53** (20130101); **G06F21/60** (20130101); **H04L9/40** (20220101); **H04L67/10** (20220101)

U.S. Cl.:

CPC **H04L9/3234** (20130101); **G06F21/53** (20130101); **G06F21/606** (20130101); **H04L63/061** (20130101); **H04L63/0853** (20130101); **H04L63/0876** (20130101); **H04L67/10** (20130101)

Field of Classification Search

CPC: H04L (63/061); H04L (63/0853); H04L (63/0876); H04L (67/10); H04L (9/3234); G06F (21/53); G06F (21/606); G06F (21/57); G06F (21/6209); G06F (9/544)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
6216175	12/2000	Sliger et al.	N/A	N/A
6260190	12/2000	Ju	N/A	N/A
6470450	12/2001	Langford	713/182	G06F 21/602
6915510	12/2004	Tock et al.	N/A	N/A
6948164	12/2004	Tinker	N/A	N/A
7178021	12/2006	Hanna	713/150	H04L 9/0825
7590738	12/2008	Mendonca	N/A	N/A
7716467	12/2009	Deffet	713/153	H04L 63/0272
7725890	12/2009	Kawaura	N/A	N/A
7765579	12/2009	Wallace	N/A	N/A
7779394	12/2009	Homing et al.	N/A	N/A
7788708	12/2009	Doane	713/168	G06F 21/31
8225308	12/2011	Sedukhin et al.	N/A	N/A
8286241	12/2011	Yeo	726/22	H04L 63/101
8473724	12/2012	Kenville et al.	N/A	N/A
8595810	12/2012	Ben Ayed	713/168	H04L 63/0815
8607200	12/2012	Kunze	N/A	N/A
8818946	12/2013	Foti	707/769	H04W 4/70
8825663	12/2013	Mahaniok	707/948	G06F 8/60
8869235	12/2013	Qureshi	713/153	H04L 67/10
8887141	12/2013	Day et al.	N/A	N/A

8898620	12/2013	Eizenman et al.	N/A	N/A
9002018	12/2014	Wilkins	713/168	H04L 9/0891
9009790	12/2014	Wuthnow	713/168	H04L 67/306
9037870	12/2014	Zheng	380/42	H04L 9/0891
9055063	12/2014	Dwan	N/A	H04L 63/0892
9111105	12/2014	Barton	N/A	H04L 51/08
9111276	12/2014	Haller	N/A	N/A
9112854	12/2014	Bhimanaik	N/A	H04L 63/0823
9208328	12/2014	Russello et al.	N/A	N/A
9224001	12/2014	Ayyalasomayajula	N/A	H04W 8/245
9246690	12/2015	Roth	N/A	G06F 21/53
9286477	12/2015	Sobel	N/A	G06F 21/606
9319219	12/2015	Quinlan	N/A	H04L 9/0822
9354849	12/2015	Debate et al.	N/A	N/A
9355060	12/2015	Barber et al.	N/A	N/A
9515999	12/2015	Ylonen	N/A	H04L 63/101
9529996	12/2015	Qureshi	N/A	H04W 12/06
9626525	12/2016	Momchilov	N/A	H04L 9/0863
9635001	12/2016	Kaseda	N/A	H04L 9/14
9866382	12/2017	Wagner	N/A	H04L 9/0822
9940454	12/2017	Richardson	N/A	H04L 63/14
9954834	12/2017	Quinlan	N/A	H04L 63/0853
10031783	12/2017	Jalagam et al.	N/A	N/A
10044507	12/2017	Raduchel	N/A	H04L 51/00
10205710	12/2018	Mitchell	N/A	H04L 63/0823
10225287	12/2018	Aamir et al.	N/A	N/A
10298880	12/2018	Okuyama	N/A	H04N 7/15
10303879	12/2018	Potlapally	N/A	G06F 21/57
10341327	12/2018	Kola	N/A	H04L 63/0823
10467421	12/2018	Aamir	N/A	H04L 63/0435
10567959	12/2019	Studerus	N/A	H04W 4/50
10599409	12/2019	Seibel et al.	N/A	N/A
10797904	12/2019	Dixon	N/A	H04N 21/6587
11228569	12/2021	Westra	N/A	H04L 63/0823

11424931	12/2021	Seibel et al.	N/A	N/A
11637703	12/2022	Raduchel	709/204	H04L 65/403
11799683	12/2022	Dixon	N/A	H04N 21/441
2002/0073398	12/2001	Tinker	N/A	N/A
2002/0087956	12/2001	Darlet	N/A	N/A
2003/0033521	12/2002	Sahlbach	726/26	H04L 9/3263
2003/0079120	12/2002	Hearn	709/229	G06F 21/10
2004/0015927	12/2003	Haber et al.	N/A	N/A
2004/0237067	12/2003	Sun et al.	N/A	N/A
2005/0081029	12/2004	Thornton	713/156	H04L 9/3226
2005/0182966	12/2004	Pham	726/5	G06F 21/606
2005/0262568	12/2004	Hansen	713/193	G06F 21/6218
2006/0150256	12/2005	Fanton	726/27	H04L 63/10
2006/0161768	12/2005	Baissus et al.	N/A	N/A
2007/0006163	12/2006	Aoki et al.	N/A	N/A
2007/0074169	12/2006	Chess et al.	N/A	N/A
2007/0271459	12/2006	Gomez	713/171	H04L 9/3073
2008/0010458	12/2007	Holtzman	713/189	H04L 63/0823
2008/0010685	12/2007	Holtzman	726/28	G06F 12/1483
2008/0059787	12/2007	Hohenberger	713/153	H04L 9/3073
2008/0083021	12/2007	Doane	726/5	G06F 21/31
2008/0184225	12/2007	Fitzgerald et al.	N/A	N/A
2009/0037921	12/2008	Simpson et al.	N/A	N/A
2009/0089766	12/2008	Day et al.	N/A	N/A
2009/0094596	12/2008	Kuiper et al.	N/A	N/A
2009/0119653	12/2008	Kettley et al.	N/A	N/A
2009/0290712	12/2008	Henry et al.	N/A	N/A
2009/0328067	12/2008	Srinivasan et al.	N/A	N/A
2010/0037100	12/2009	Lopian	N/A	N/A
2010/0138652	12/2009	Sela	713/158	H04L 9/3268
2010/0287541	12/2009	Saunders	717/139	G06F 11/3466
2010/0303236	12/2009	Laaksonen	380/283	H04W 12/50
2010/0333079	12/2009	Sverdlov et al.	N/A	N/A
2011/0058673	12/2010	Zheng	380/255	H04L 9/302
2011/0066999	12/2010	Rabinovich et al.	N/A	N/A
2011/0087692	12/2010	Masone	709/224	G06F 9/5027
2011/0154031	12/2010	Banerjee	380/278	H04L 63/20

2012/0023487	12/2011	Letca et al.	N/A	N/A
2012/0036554	12/2011	Golan et al.	N/A	N/A
2012/0079609	12/2011	Bender	726/30	H04L 63/105
2012/0124559	12/2011	Kondur	717/125	G06F 8/41
2012/0124659	12/2011	Craft	726/9	G06Q 20/204
2012/0159163	12/2011	von Behren	713/168	G06Q 20/3823
2012/0162538	12/2011	Dixon	348/734	H04N 21/2393
2012/0204032	12/2011	Wilkins	713/170	H04L 9/321
2012/0209923	12/2011	Mathur	709/229	G06F 21/53
2012/0210443	12/2011	Blaisdell	726/27	H04W 12/37
2012/0291103	12/2011	Cohen	726/4	G06F 21/6218
2012/0328105	12/2011	Mukkara	380/277	H04L 9/0897
2012/0331283	12/2011	Chandran	713/150	H04L 9/088
2013/0013555	12/2012	Foti	707/E17.127	H04W 4/70
2013/0024383	12/2012	Kannappan	705/71	H04L 9/321
2013/0042222	12/2012	Maddela	N/A	N/A
2013/0054801	12/2012	Belchee	709/225	G06F 21/105
2013/0073607	12/2012	Smirnov	709/203	G06Q 10/00
2013/0091543	12/2012	Wade	726/1	G06F 21/53
2013/0097659	12/2012	Das	726/1	G06F 21/51
2013/0097660	12/2012	Das	726/1	H04L 63/205
2013/0212388	12/2012	D'Souza et al.	N/A	N/A
2013/0227280	12/2012	Quinlan	713/165	H04L 9/0822
2013/0239192	12/2012	Linga	726/3	H04L 63/061
2013/0276056	12/2012	Epstein	N/A	N/A
2013/0283245	12/2012	Black et al.	N/A	N/A
2013/0283400	12/2012	Schneider	726/30	H04W 12/06
2013/0326590	12/2012	Schroeder, Jr.	726/4	H04L 63/08
2014/0006773	12/2013	Chazalet	713/150	H04L 63/0464
2014/0007182	12/2013	Qureshi	726/1	G06F 21/6218
2014/0013316	12/2013	Kemmler	717/170	G06F 8/65
2014/0032691	12/2013	Barton	709/206	H04W 12/08
2014/0032759	12/2013	Barton	709/225	H04W 12/08
2014/0040979	12/2013	Barton	726/1	H04W 12/30
2014/0059525	12/2013	Jawa et al.	N/A	N/A
2014/0068273	12/2013	Sobel	713/189	G06F 16/196
2014/0068593	12/2013	McErlane	717/176	G06F 8/65

2014/0090077	12/2013	Jeong	726/26	G06F 21/51
2014/0095874	12/2013	Desai	713/168	H04L
2014/0098449	12/2013	Hellberg et al.	N/A	63/0815
2014/0108558	12/2013	Borzycki	709/205	N/A
2014/0122880	12/2013	Krishnaswamy	713/168	H04L 63/10
2014/0123207	12/2013	Agarwal	726/1	H04L
2014/0165049	12/2013	Diamos et al.	N/A	63/126
2014/0177839	12/2013	Wagner	380/259	G06F
2014/0181518	12/2013	Kim	713/168	21/6218
2014/0181522	12/2013	Tanizawa	713/171	N/A
2014/0181801	12/2013	Voronkov et al.	N/A	N/A
2014/0250511	12/2013	Kendall	726/6	G06F 21/60
2014/0282446	12/2013	Debate	717/145	G06F 9/54
2014/0282901	12/2013	Dwan	726/4	H04L 63/06
2015/0040246	12/2014	Yuen	726/30	N/A
2015/0046712	12/2014	Korkishko	713/170	G06F 21/44
2015/0169877	12/2014	Mahaffey	726/22	G06F 8/41
2015/0172060	12/2014	Mahaffey	713/176	H04L
2015/0172146	12/2014	Mahaffey	709/224	67/535
2015/0222604	12/2014	Ylonen	713/171	H04L
2015/0319144	12/2014	Barton	713/168	9/3234
2015/0326567	12/2014	Hamburg et al.	N/A	G06F
2016/0072629	12/2015	Kulkarni	713/176	21/577
2016/0119292	12/2015	Kaseda	713/165	G06F 21/57
2016/0191499	12/2015	Momchilov	713/171	G06F 21/44
2016/0191645	12/2015	Hayton	709/203	H04L
2016/0283406	12/2015	Linga	N/A	63/045
2016/0308845	12/2015	Quinlan	N/A	G06F 21/78
2017/0012953	12/2016	Ylonen	N/A	H04L
2017/0019386	12/2016	Ylonen	N/A	63/0428
2017/0019387	12/2016	Ylonen	N/A	H04L
2017/0041349	12/2016	Ylonen	N/A	9/0891
2017/0055101	12/2016	Studerus	N/A	H04L
2017/0063975	12/2016	Prakash	N/A	63/0435
2017/0094509	12/2016	Mistry	N/A	H04L 9/321
				H04W 12/35
				G06F 21/73
				H04W 4/50

2017/0116424	12/2016	Aamir	N/A	G06F 21/606
2017/0147810	12/2016	Richardson	N/A	H04W 12/37
2017/0163689	12/2016	Ylonen	N/A	H04L 63/065
2017/0171175	12/2016	Ylonen	N/A	H04L 63/101
2017/0222995	12/2016	Ylonen	N/A	H04L 9/321
2017/0257345	12/2016	Westra	N/A	H04W 12/069
2017/0366580	12/2016	Ylonen	N/A	H04L 63/065
2017/0366581	12/2016	Ylonen	N/A	G06F 21/575
2018/0025180	12/2017	Wang	726/1	G06F 21/6281
2018/0183595	12/2017	Raduchel	N/A	H04L 63/102
2019/0005229	12/2018	Hlaing	N/A	G06F 9/445
2019/0075134	12/2018	Ylonen	N/A	H04L 63/0442
2020/0089869	12/2019	Richardson	N/A	H04W 12/122

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
1942845	12/2006	CN	N/A
101971186	12/2010	CN	N/A
103347020	12/2012	CN	N/A
104392188	12/2014	CN	N/A
104965732	12/2014	CN	N/A
2746981	12/2013	EP	N/A
2743855	12/2016	EP	N/A
2010029362	12/2009	WO	N/A
2014052917	12/2013	WO	N/A
2015196967	12/2014	WO	N/A

OTHER PUBLICATIONS

Canadian Office Action; Application No. 2,955,585; Jan. 9, 2023; 3 pages. cited by applicant

Canadian Office Action; Application No. 2,956,103; Feb. 10, 2023; 5 pages. cited by applicant

Canadian Office Action; Application No. 2,955,585; Jul. 18, 2023; 4 pages. cited by applicant

An, Lianjun, et al.; “Effective Workforce Lifecycle Management via System Dynamics Modeling and Simulation”; IEEE; Proceedings of the 2007 Winter Simulation Conference; Dec. 9-12, 2007; 9 pages. cited by applicant

Etkin, Joshua, et al.; “Development Life Cycle of Computer Networks: The Executable Model Approach”; IEEE Transactions on Software Engineering; vol. 15, No. 9; Sep. 1989; 12 pages. cited by applicant

McCalla, Clement, et al.; “A Time-Dependent Queueing-Network Model to Describe the Life-Cycle Dynamics of Private-Line Telecommunication Services”; Telecommunication Systems; vol.

19, No. 1; Jan. 2002; 30 pages. cited by applicant

Narayan, Vish; "Application Integration Environment for Messaging/Queueing Model"; IEEE; Proceedings ISADS 95. Second International Symposium on Autonomous Decentralized Systems; Apr. 25-27, 1995; 6 pages. cited by applicant

IP.com; "IQQueryQuickExport—202004300627"; Apr. 30, 2020; 4 pages. cited by applicant

IP.com; "IQQueryQuickExport—202010010647"; Oct. 7, 2020; 4 pages. cited by applicant

IP.com; "IQQueryQuickExport—202109250251"; Sep. 28, 2021; 5 pages. cited by applicant

IP.com; "IQQueryQuickExport"; Apr. 21, 2022; 4 pages. cited by applicant

Office Action dated Jan. 7, 2015; U.S. Appl. No. 14/208,068, filed Mar. 13, 2014; 14 pages. cited by applicant

Final Office Action dated Jul. 28, 2015; U.S. Appl. No. 14/208,068, filed Mar. 13, 2014; 12 pages. cited by applicant

Advisory Action dated Nov. 30, 2015; U.S. Appl. No. 14/208,068, filed Mar. 13, 2014; 3 pages. cited by applicant

Notice of Allowance dated Jan. 29, 2016; U.S. Appl. No. 14/208,068, filed Mar. 13, 2014; 18 pages. cited by applicant

Office Action dated Oct. 4, 2017; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 51 pages. cited by applicant

Final Office Action dated Apr. 24, 2018; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 35 pages. cited by applicant

Advisory Action dated Jun. 18, 2018; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 5 pages. cited by applicant

Office Action dated Nov. 27, 2018; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 52 pages. cited by applicant

Final Office Action dated Jun. 13, 2019; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 37 pages. cited by applicant

Advisory Action dated Aug. 27, 2019; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 4 pages. cited by applicant

Office Action dated Nov. 12, 2019; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 41 pages. cited by applicant

Final Office Action dated May 5, 2020; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 38 pages. cited by applicant

Advisory Action dated Jul. 27, 2020; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 4 pages. cited by applicant

Office Action dated Oct. 7, 2020; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 37 pages. cited by applicant

Final Office Action dated Apr. 14, 2021; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 39 pages. cited by applicant

Advisory Action dated Jun. 28, 2021; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 5 pages. cited by applicant

Office Action dated Oct. 8, 2021; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 46 pages. cited by applicant

Notice of Allowance dated Apr. 21, 2022; U.S. Appl. No. 15/007,547, filed Jan. 27, 2016; 12 pages. cited by applicant

Office Action dated Aug. 2, 2017; U.S. Appl. No. 15/013,438, filed Feb. 2, 2016; 32 pages. cited by applicant

Final Office Action dated May 11, 2018; U.S. Appl. No. 15/013,438, filed Feb. 2, 2016; 35 pages. cited by applicant

Advisory Action dated Jul. 26, 2018; U.S. Appl. No. 15/013,438, filed Feb. 2, 2016; 5 pages. cited by applicant

Office Action dated Jan. 14, 2019; U.S. Appl. No. 15/013,438, filed Feb. 2, 2016; 26 pages. cited by applicant
Notice of Allowance dated Jul. 29, 2019; U.S. Appl. No. 15/013,438, filed Feb. 2, 2016; 15 pages. cited by applicant
Notice of Allowance dated Nov. 7, 2019; U.S. Appl. No. 15/013,438, filed Feb. 2, 2016; 12 pages. cited by applicant
PCT International Search Report; Application No. PCT/US2014/025397; Aug. 7, 2014; 2 pages. cited by applicant
PCT Written Opinion of the International Searching Authority; Application No. PCT/US2014/025397; Aug. 7, 2014; 6 pages. cited by applicant
Chinese Office Action Application No. 201710057013.6; Jul. 5, 2021; 14 pages. cited by applicant
European Extended Search Report; Application No. 17152587.6; Jun. 23, 2017; 7 pages. cited by applicant
European Examination Report; Application No. 17152587.6; Apr. 5, 2019; 6 pages. cited by applicant
European Examination Report; Application No. 17152587.6; Mar. 30, 2020; 6 pages. cited by applicant
European Extended Search Report; Application No. 17153148.6; May 22, 2017; 8 pages. cited by applicant

Primary Examiner: Lynch; Sharon S

Attorney, Agent or Firm: Conley Rose, P.C.

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS (1) This application is a continuation of U.S. patent application Ser. No. 15/007,547 filed Jan. 27, 2016 by James Seibel, et al. entitled, “Trusted Execution Environment”, which is incorporated by reference herein as if reproduced in its entirety.

BACKGROUND

(1) Computer security, particularly to safeguard the proprietary and sensitive data of corporations and other enterprises, has become increasingly scrutinized as more and more data travels electronically over public access networks. An enterprise or other organization, however, with many employees or members presents a particular vulnerability, as any user may be leveraged to obtain access. Further, such enterprises may employ a common set of applications on computing devices of each individual user. It is problematic to mandate and enforce exclusive corporate usage of these personal devices; as a practical matter, little control can be exercised over a machine in a user/employee's possession. Additional applications and activities undertaken by users can interfere with and/or compromise enterprise activities on the computing device due to common execution environments shared by the enterprise apps and user supplied apps, resulting in a vulnerability that rests upon the integrity of extraneous apps loaded by the user.

SUMMARY

(2) A trusted execution environment on an enterprise-enabled computing device, whether owned by the enterprise or the user/employee, hereinafter called an “enterprise computing device”, allows invocation of a suite of secure enterprise applications (apps) on a device without hindering external or non-enterprise apps from running on the same enterprise computing device. Each of the enterprise computing devices executes the suite of secure enterprise apps, which interact with each

other in a trusted manner such that outside apps or network connections are prevented from accessing the enterprise apps. The enterprise computing device, however, also executes non enterprise applications which operate independently of the enterprise apps in the same address space, and which can communicate with other non-enterprise apps, and using the same unmodified operating system as the enterprise apps on the enterprise computing device. The trusted execution environment therefore restricts interprocess communication only between the set of enterprise apps and non-enterprise apps, and also permits unimpeded operation of other apps under the same OTS (off the shelf) operating system.

(3) Configurations herein are based, in part, on the observation that corporations and other enterprises often employ a dedicated suite of applications or apps for handling proprietary data. Employees or members of the enterprise desire to invoke the enterprise application suite on a portable computing device along with other apps that are not directed to proprietary data. Unfortunately, conventional approaches to providing a secure or trusted application space, such as for executing the enterprise apps, suffer from the shortcoming that changes are required to the native operating system (OS) or conditionally assessing attempted memory accesses or memory partitions. Usage of a modified or custom OS presents cost and logistical issues with deployment, as well as compatibility issues with execution of other apps that are not in the dedicated suite.

(4) Accordingly, configurations herein substantially overcome the above-described shortcomings by providing a trusted application space on a portable computing device for concurrent execution of trusted apps with other user selected or general apps with an unmodified off-the-shelf (OTS) OS for maintaining compatibility and avoiding deployment of a custom, trusted operating system.

(5) In further detail, configurations herein provide a trusted execution environment under an unmodified, off-the-shelf (OTS) operating system on a mobile computing device by loading a plurality of applications into the memory of an execution environment on the computing device. The execution environment on the computing device has memory, a processor, a file system and a network interface, and communicates securely by designating a communication association between one of the loaded applications and a related application of the plurality of loaded applications. The method detects, by any of the loaded applications, an attempt to communicate with a recipient application of the loaded applications defining a trusted set or suite of apps. The attempted communication is disallowed if the applications do not have a designated communication association. Implementation on an unmodified, OTS OS allows loading one or more other applications into the execution environment, such that the other applications that are not designated as trusted are prevented from communicating with any of the trusted applications.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) The foregoing and other objects, features and advantages of the invention will be apparent from the following description of particular embodiments of the invention, as illustrated in the accompanying drawings in which like reference characters refer to the same parts throughout the different views. The drawings are not necessarily to scale, emphasis instead being placed upon illustrating the principles of the invention.

(2) FIG. 1 is a prior art system for controlling application memory access;

(3) FIG. 2 is a context diagram of an enterprise environment suitable for use with configurations herein;

(4) FIG. 3 is an architecture diagram of a computing device in the environment in FIG. 2;

(5) FIG. 4 shows a block diagram of the computing device in FIG. 3 according to configurations herein; and

(6) FIGS. 5A and 5B are a flowchart of application (app) execution in the computing device of

FIG. 4.

DETAILED DESCRIPTION

(7) Depicted below is an example of an enterprise having a plurality of portable computing devices, as might be used by employees of the enterprise, each exhibiting a secure execution environment. Examples of communication between apps in the secure execution environment depict particular features, however should not be taken to limit approaches of the invention as claimed herein.

Various organizational settings may elect computing devices having a secure execution environment in a stationary, mobile, portable, handheld or other physical configuration.

(8) The applications, or apps, as described represent an executable entity launched in the memory space of a computing device. The computing devices have an operating system such as Android™, IOS (iPhone® Operating System), WINDOWS®, Linux or other suitable platform, however it should be emphasized that the techniques disclosed herein are independent of the native OS and do not impose or require changes to the underlying OS of the computing device. The disclosed approach also employs off-the-shelf applications, such that loaded apps do not require a specialized version for compatibility with the disclosed approach.

(9) An example configuration depicted herein provides an environment for secure applications to communicate with each other in the presence of other insecure apps. This is implemented within the applications themselves, and therefore does not rely on the operating system to provide a special trusted container. Individual processes in the trusted execution environment are wrapped to provide interception of system calls which affect communication with and invocation of other processes.

(10) The trusted execution environment is overseen by a monitor process, or “keystore” application that manages communication between secure applications. Encryption keys are managed and distributed via the keystore application. The keystore application has an Access Control List (ACL) or other repository that indicates which applications can communicate with each other. Each application has a unique ID that is mapped to a list of IDs of other applications. A trusted application can only communicate with an application that is in its ACL.

(11) When an application is installed, the keystore application generates a new encryption key for it. Various key generation mechanisms such as Diffie-Hellman are used to establish a trusted inter-process communication channel between the newly installed application and the keystore application (KA) in order to transfer the key. An application that wants to communicate with another application generally has two options. Either a direct communication channel can be established, or the app can write a file to disk that is read by another.

(12) For example, when an application A tries to read a file created by a different application B, it reads the file's key ID from plaintext file metadata, then queries the KA for the correct decryption key. If application A's unique ID does not have an ACL entry for Application B, the KA denies access. If it has access, the decryption key is returned over the secure inter-process communication channel.

(13) When an application wants to communicate directly, a secure IPC channel between the two is established using AES GCM encryption using a common key retrieved from the KA or another suitable encryption mechanism. In this manner, secure sharing of encrypted files and IPC communications can occur between the trusted apps, managed by the keystore application that limits access.

(14) FIG. 1 is an example of a prior art system for controlling application memory access.

Referring to FIG. 1, a conventional computer 10 for operating a trusted process or address space maintains partitions or regions of memory 12 designated as “trusted.” A CPU 30 directs memory accesses responsive to the OS 26, which occupies a high or low address region of memory (high address region in the example shown). An untrusted portion 20 maintains no particular supervisory control. A trusted memory portion 24 occupies the region below the OS 26, and a free portion 22 is allocated as processes in the untrusted 20 and trusted 24 portions grow towards each other.

(15) The conventional operating system **26**, before permitting a memory access by the CPU, checks the access to ensure it does not violate the trusted memory region **24**. A memory access **40** from a process in the untrusted portion **20** to another process in the untrusted region **20** is allowed, as is a memory access **42** from the trusted region **24** to the trusted region **24**. However, an access attempt **44** from the untrusted portion **20** to the trusted region **24** is disallowed, as shown by the “X” on purported access **44**. The operating system, however, is required to take an integral role in scrutinizing each memory access to ensure the trusted memory portion **24** remains inaccessible to the untrusted portion **20**. This generally requires a specialized operating system, or at least modifications to a standard operating system, for scrutinizing each memory access. Runtime performance implications are also imposed as each memory access triggers a range check. It would be beneficial to provide a trusted execution environment which allows trusted processes to communicate with other trusted processes, or apps, on an unmodified, vendor supplied OTS OS.

(16) FIG. 2 is a context diagram of an enterprise environment suitable for use with configurations herein. Referring to FIG. 2, in an enterprise setting **100** such as a corporation or business having employees using a particular set or suite of apps for handling proprietary, or corporate sensitive, information invokes a number of portable computing devices **110-1 . . . 110-N** (**110** generally) for each employee. The computing devices **110** may be tablets, laptops, handheld or phone type devices, or may even be more stationary devices such as desktop computers. Each of the computing devices **110** executes apps **150-1 . . . 150-N** (**150** generally), which send messages **140-1 . . . 140-N** (**140** generally) to other apps **150** on the same or other computing devices **110**, which may be remotely accessible over a public infrastructure network **132** such as the Internet. Each of the apps **150** is generally defined by a process running under the OS, however more complex apps may include more than one process. Messages **140** deemed trusted (**140-1 . . . 140-4**), as will be discussed further below, are permitted to be received by a recipient app, while untrusted messages **140-5** are disallowed.

(17) FIG. 3 is an architecture diagram of a computing device **110** in the environment in FIG. 2. The computing device **110** typically includes a processor **130**, memory **112**, a file system **114** for accessing a storage volume **116**, and a network interface **118** for accessing the network **132**. Each of the computing devices **110** employs a trusted execution environment **160** for launching and executing the apps **150**. The trusted execution environment **160** includes a set of processes, or apps, **150-10 . . . 150-12** in this example, including a wrapping layer **152** for monitoring messages and other accesses invoked by the process **150**, as disclosed in further detail in copending U.S. patent application Ser. No. 14/208,068, filed Mar. 13, 2014, entitled “MODIFICATION OF COMPILED APPLICATIONS AND APPLICATION MANAGEMENT USING RETRIEVABLE POLICIES,” incorporated herein by reference. Each of the processes **150** in the trusted group **150-N** may communicate with other processes in the trusted group **150-N**, as shown by message paths **154**. Untrusted processes **153** outside the trusted group **150-N** may not communicate with the trusted processes **150**, as shown by blocked paths **155**. The trusted execution environment **160** identifies an attempt to communicate with any of the apps **150** in the execution environment, and disallows the communication if a corresponding communication association is not found. In this manner, any number of apps, or processes, may be concurrently loaded into the same memory space **112** for invocation by the user, within the hardware limits of the device **110**. The trusted environment disclosed herein prevents the trusted apps **150** from being accessed by any of the untrusted apps **153**, without any other restrictions on execution or memory ranges in which each group (**150**, **153**) of apps may reside.

(18) FIG. 4 shows a block diagram of the computing device in FIG. 3 according to configurations herein. Referring to FIGS. 2-4, the execution environment **160** on each of the trusted computing devices **110** includes a keystore application (KA) **170** for distributing keys for allowing interprocess communication between the processes **150** in the trusted group **150-N** of apps. The KA securely holds all of the application keys when the trusted environment is not active, e.g., when the

device is asleep or before a trusted app **150** has been started. When a trusted app **150** starts, it retrieves the corresponding file system and IPC keys from the KA. [Ed. Note: The keystore does not monitor IPC, it merely controls access to the keys that enable communication, and thereby controls which apps can communicate.]

(19) The KA **170** maintains an association table **172** for defining access control between the apps **150**. The association table **172** includes entries **174-1 . . . 174-N** (**174** generally) for denoting related apps **150** which may communicate with each other. Each entry **174** defines an app **180** and a related app **182** to which communication may be performed, denoted by app IDs or other unique identifier for identifying the app **150** on the computing device **110**. Associations may be bidirectional or unidirectional depending on how the control system is implemented in alternate configurations. To support a unidirectional association, the table **172** may define source and destination apps, for example. The association entries **174** may also include a key **184** which is employed for encrypting data transferred between the processes. In a particular configuration, an app may have a master key which is then used to compute a file specific key for writing files **216** to and reading files **216** from the file system **116**, as discussed further below.

(20) Each app **150** may generally invoke one of three modes of interprocess communication (IPC): in-memory, file transfer, or network transmission. For example, continuing to refer to FIG. **4**, the app **150-1** intends to communicate with another app **150-2** on the same device, as shown by arrow **200**. The KA **170** examines the association table **172** for an entry **174** designating the apps **150-1**, **150-2** as related, and thus permitted to communicate. Generally a match between an app ID of the apps **150-1**, **150-2** and the app **180** and related apps **182** fields is performed, however other implementation such as a list of app IDs associated with a particular app **150** may be provided.

(21) Control of in-memory IPC, which may be any suitable IPC mechanism as supported by the OS, such as shared memory or sockets, within the trusted app space can be achieved in either of two modes, a permissive mode, or a restrictive mode. In the permissive mode, when two apps **150-1** and **150-2** first want to communicate, they each request a key from the KA **170**, arrows **205** and **220** in FIG. **4**, then retain the common key in their private memory as long as each app is running. The stored key is then used to encrypt and decrypt any messages **200** between the two apps.

(22) In the restrictive mode of in-memory IPC, continuing to refer to FIG. **4**, app **150-1** initiates a message transmission to app **150-2** by a request exchange with the KA **170**, as shown by arrows **205**. The KA **170** performs a lookup in the association table **172**, and identifies key **1** corresponding to the association between apps **150-1** and **150-2**, from the key column **184**. The sending app **150-1** encrypts a message **214** with the retrieved key **1**, and writes the encrypted message **214** to an IPC channel **230** such as shared memory or other mechanism. The receiving app **150-2** queries the KA **170** for authority to receive communications from app **150-1**, and upon positive verification also receives key **1**, as shown by arrows **220**. App **150-2** reads the encrypted message **214** from the IPC channel **230**, and uses the received key **1** to decrypt the message. It should be noted that implementation of the restrictive mode of IPC control would likely have a performance impact with current hardware and OS choices.

(23) The file system **116** generally serves the entire device **110** including both the trusted apps **150** and the untrusted apps **153**. A trusted app **150** (the author app) creates a unique identifier for each file **216** that it writes to disk, and encrypts the contents of the file **216** using an algorithm that takes its master key **184** and the file identifier as inputs. The KA can provide the decryption key for the file **216** to another trusted app **150** (the recipient app) that is allowed access (per the association table **172**) since it has the author app's master key **184** and the same algorithm for creating the file key. The KA uses the author key and the file **216** identifier to generate the decryption key which it passes to the recipient app. The resulting key allows communication between the author and recipient only through that particular file. In this manner, learning a file-specific key only exposes access to that one file.

(24) When using network transmission, a trusted app **150** cannot encrypt the messages it sends or

decrypt those it receives since it cannot assume a common keystore application between it and the remote system. Referring to FIG. 4, any messages **240** the trusted app **150** sends through the public Internet **132** must be secured through use of a virtual private network (VPN) that encrypts the network traffic and is only accessible to the trusted apps and trusted remote clients or servers.

(25) FIGS. 5A and 5B are a flowchart of application (app) execution in the computing device of FIG. 4. Referring to FIGS. 2-5B, at step **500**, the method of providing a trusted execution environment **160** in a mobile computing device **110** as disclosed herein includes loading a plurality of trusted applications into the memory **112** of an execution environment **160**, in which the execution environment has memory **112**, a processor **130**, a file system **114** and a network interface **118**. In addition, at least one other (untrusted) application **153** may be loaded into the execution environment **160**, such that the one other application is prevented from communicating with any of the trusted applications **150**, as disclosed at step **501**. Both trusted **150** and untrusted **153** applications can therefore coexist in the same address region and access the same file store **114**, and execute on an unmodified OTS OS, without permitting the untrusted applications **153** to access or message the trusted applications **150**.

(26) Accordingly, in the example configuration, detecting an attempt to communicate includes wrapping the applications in the set of trusted applications, and leaving the other applications unmodified, such that the wrapped applications are operable in the execution environment **160** responsive to an unmodified operating system, as depicted at step **502**. Wrapping may be performed by any suitable method, such as upon installation in the host device, by binding with DLLs (Dynamic Link Library), upon loading in preparation for launch, or other suitable mechanism, as disclosed more fully in the copending application cited above.

(27) The execution environment **160** designates a communication association between one of the trusted applications and a related application of the plurality of trusted applications, as shown at step **510**. In the example configuration, this includes storing the designated communication associations in an association table accessible by the KA **170**, such that the KA **170** is configured to receive an identity of a trusted application **150**, and indicate if a communication association exists with another application **150** in the execution environment **160**, as depicted at step **511**. The association table **172** includes a set of entries **174**, in which each entry has an indication of one of the applications of the plurality of the applications and an identity of the related application to which the application may communicate, as shown at step **512**. Alternate representations, such as lists, may also be employed.

(28) The application wrapper **152** controls access to system calls and other invocations which could be used to access or transmit sensitive data from the wrapped app **150**. The wrapper **152** detects an attempt to communicate by identifying file accesses, IPC accesses and network accesses for each of the trusted applications, such that the trusted applications **150-N** define a set of trusted applications operable for communication only with other applications in the set of trusted applications exclusive of other applications in the memory (i.e. untrusted apps **153**) outside the set of trusted applications.

(29) A check is performed, at step **522**, to determine if a communication association exists between the apps **150** in questions. If no match is found, than at least one of the apps **150** is not trusted, or in the trusted set. The KA **170** disallows the attempted communication if the applications do not have a designated communication association, as depicted at step **530**. No keys are sent from the KA, removing an ability for the purported recipient app to interpret the transmitted message.

(30) Otherwise, if the check at step **522** indicates that a communication association exists, the sending app identifies a message for transmission between the associated applications. The KA **170** generates a key based on the identity of the related application to which communication is attempted, as depicted at step **550**. The sending app **150** transmits, using the computed key, the identified message to the recipient application **150**, as shown at step **560**.

(31) If the transmission mode was via file transfer, then transmitting further includes writing the identified message to the file **216**, such that the contents of the file **216** are encrypted using the

generated key, as disclosed at step 561, and transmitting the generated key to the recipient application from the KA 170, such that the KA 170 is operable to store and retrieve the designated communication associations, as depicted at step 562.

(32) The recipient app 150 receives the computed key at the recipient application for decrypting the generated message, as shown at step 563 reads the file 216 and decrypts the message using the transmitted key, as disclosed at step 564.

(33) It will be appreciated by those skilled in the art that alternate configurations of the disclosed invention include a multiprogramming or multiprocessing computerized device such as a workstation, handheld or laptop computer or dedicated computing device or the like configured with software and/or circuitry (e.g., a processor as summarized above) to process any or all of the method operations disclosed herein as embodiments of the invention. Still other embodiments of the invention include software programs such as a Java Virtual Machine and/or an operating system that can operate alone or in conjunction with each other with a multiprocessing computerized device to perform the method embodiment steps and operations summarized above and disclosed in detail below. One such embodiment comprises a computer program product that has a computer-readable storage medium including computer program logic encoded thereon that, when performed in a multiprocessing computerized device having a coupling of memory and a processor, programs the processor to perform the operations disclosed herein as embodiments of the invention to carry out data access requests. Such arrangements of the invention are typically provided as software, code and/or other data (e.g., data structures) arranged or encoded on a non-transitory computer readable storage medium such as an optical medium (e.g., CD-ROM), floppy or hard disk or other medium such as firmware or microcode in one or more ROM, RAM or PROM chips, field programmable gate arrays (FPGAs) or as an Application Specific Integrated Circuit (ASIC). The software or firmware or other such configurations can be installed onto the computerized device (e.g., during operating system execution or during environment installation) to cause the computerized device to perform the techniques explained herein as embodiments of the invention.

(34) While the system and methods defined herein have been particularly shown and described with references to embodiments thereof, it will be understood by those skilled in the art that various changes in form and details may be made therein without departing from the scope of the invention encompassed by the appended claims.

Claims

1. A method of providing an execution environment in a computing device, comprising: loading a plurality of trusted applications into a memory of the computing device; maintaining, in the memory, a plurality of communication associations; detecting an attempt to communicate with a recipient application by an originating application, the recipient application being selected from the plurality of trusted applications; searching through the plurality of communication associations for a communication association comprising an identifier of the recipient application and an identifier of the originating application; when the communication association comprising the identifier of the recipient application and the identifier of the originating application is not found, disallowing the communication; and when the communication association comprising the identifier of the recipient application and the identifier of the originating application is found, identifying a message for transmission between the originating application and the recipient application, generating a key based on the identifier of the recipient application and a unique file-specific identifier of a file system, writing the identified message to a file, the contents of the file encrypted using the generated key; transmitting, using the generated key, the file with the identified message to the recipient application, receiving the generated key at the recipient application for decrypting the message, and reading the file and decrypting the message, by the recipient application, using the generated key.

2. The method of claim 1 further comprising loading at least one untrusted application into the execution environment, the at least one untrusted application being prevented from communicating with any of the trusted applications.
3. The method of claim 1 wherein the transmitting comprises: transmitting the generated key to the recipient application from a keystore application, the keystore application operable to store and retrieve the plurality of communication associations.
4. The method of claim 1 wherein the transmitting comprises: designating an IPC (Interprocess Communication) channel between a transmitting application of the trusted applications and the recipient application; encrypting the identified message using the generated key; transmitting the encrypted message to the recipient application from the originating application; and transmitting the generated key to the recipient application from a keystore application, the keystore application operable to store and retrieve the plurality of communication associations.
5. The method of claim 1 wherein detecting the attempt to communicate identifies file accesses, IPC accesses and network accesses for each of the trusted applications, the trusted applications defining a set of trusted applications operable for communication only with other applications in the set of trusted applications exclusive of other applications in the memory outside the set of trusted applications.
6. The method of claim 5 wherein detecting an attempt to communicate further comprises wrapping the applications in the set of trusted applications, and leaving the other applications in the memory outside the set of trusted applications unmodified, the wrapped applications operable in the execution environment responsive to an unmodified operating system.
7. The method of claim 1 further comprising storing designated communication associations of one or more of the trusted applications in an association table accessible by a keystore application, the keystore application configured to receive an identifier of a trusted application and indicate if a communication association exists with another application in the execution environment.
8. The method of claim 7 wherein the association table includes entries, each entry having an indication of one of the plurality of the trusted applications and an identifier of one or more applications to which the one of the plurality of trusted applications may communicate.
9. A computing device for providing an execution environment, comprising: a memory; and a processor; wherein the memory and the processor cooperate to: load a plurality of trusted applications into the memory; maintain, in the memory, a plurality of communication associations; detect an attempt to communicate with a recipient application by an originating application, the recipient application being selected from the plurality of trusted applications; searching through the plurality of communication associations for a communication association comprising an identifier of the recipient application and an identifier of the originating application; when the communication association comprising the identifier of the recipient application and the identifier of the originating application is not found, disallowing the communication; and when the communication association comprising the identifier of the recipient application and the identifier of the originating application is found, identifying a message for transmission between the originating application and the recipient application, generating a key based on the identifier of the recipient application and a unique file-specific identifier of a file system, writing the identified message to a file, the contents of the file encrypted using the generated key; transmitting, using the generated key, the file with the identified message to the recipient application, and receiving the generated key at the recipient application for decrypting the message, and reading the file and decrypting the message, by the recipient application, using the generated key.
10. The computing device of claim 9, wherein the memory and the processor further cooperate to load at least one untrusted application into the execution environment, the at least one untrusted application being prevented from communicating with any of the trusted applications.
11. The computing device of claim 9 wherein the transmitting comprises: transmitting the generated key to the recipient application from a keystore application, the keystore application

operable to store and retrieve the plurality of communication associations.

12. The computing device of claim 9 wherein the transmitting comprises: designating an IPC (Interprocess Communication) channel between a transmitting application of the trusted applications and the recipient application; encrypting the identified message using the generated key; transmitting the encrypted message to the recipient application from the originating application; and transmitting the generated key to the recipient application from a keystore application, the keystore application operable to store and retrieve the plurality of communication associations.

13. The computing device of claim 9 wherein detecting the attempt to communicate identifies file accesses, IPC accesses and network accesses for each of the trusted applications, the trusted applications defining a set of trusted applications operable for communication only with other applications in the set of trusted applications exclusive of other applications in the memory outside the set of trusted applications.

14. The computing device of claim 13 wherein detecting an attempt to communicate further comprises wrapping the applications in the set of trusted applications, and leaving the other applications in the memory outside the set of trusted applications unmodified, the wrapped applications operable in the execution environment responsive to an unmodified operating system.

15. The computing device of claim 9, wherein the memory and the processor further cooperate to store designated communication associations of one or more of the trusted applications in an association table accessible by a keystore application, the keystore application configured to receive an identifier of a trusted application and indicate if a communication association exists with another application in the execution environment.

16. The computing device of claim 15 wherein the association table includes entries, each entry having an indication of one of the plurality of the trusted applications and an identifier of one or more applications to which the one of the plurality of trusted applications may communicate.

17. A non-transitory computer readable medium having stored thereon executable code for execution by a processor of a computing device, the executable code comprising instructions for: loading a plurality of trusted applications into a memory of the computing device; maintaining, in the memory, a plurality of communication associations; detecting an attempt to communicate with a recipient application by an originating application, the recipient application being selected from the plurality of trusted applications; searching through the plurality of communication associations for a communication association comprising an identifier of the recipient application and an identifier of the originating application; when the communication association comprising the identifier of the recipient application and the identifier of the originating application is not found, disallowing the communication; and when the communication association comprising the identifier of the recipient application and the identifier of the originating application is found, identifying a message for transmission between the originating application and the recipient application, generating a key based on the identifier of the recipient application and a unique file-specific identifier of a file system, writing the identified message to a file, the contents of the file encrypted using the generated key; transmitting, using the generated key, the file with the identified message to the recipient application, and receiving the generated key at the recipient application for decrypting the message, and reading the file and decrypting the message, by the recipient application, using the generated key.

18. The non-transitory computer readable medium of claim 17, wherein the executable code comprising instructions for storing designated communication associations of one or more of the trusted applications in an association table accessible by a keystore application, the keystore application configured to receive an identifier of a trusted application and indicate if a communication association exists with another application in the execution environment.

19. The non-transitory computer readable medium of claim 18, wherein each communication association of the plurality of communication associations comprises an entry of the associations

table, the entry comprising both a first identifier for a first application and a second identifier for a second application, the first application and the second application being selected from the plurality of trusted applications.
